

Práctica de JavaScript - Manipulación del DOM y Eventos

Índice

1. [Ejercicios de Eventos y data-attributes](#)
 2. [Ejercicios de Propagación y Delegación de Eventos](#)
 3. [Ejercicios de Manipulación del DOM](#)
 4. [Ejercicios Adicionales Propuestos](#)
 5. [Preguntas Frecuentes](#)
-

Ejercicios de Eventos y data-attributes

Ejercicio 1: Identificar Elementos con dataset.id

Problema: Aprende a identificar elementos HTML dinámicos con dataset.id usando eventos en JavaScript.

Actividad:

1. Crea una lista de botones en HTML, y a cada botón asígnale un atributo data-id único.
2. Al hacer clic en cualquiera de los botones, muestra una alerta que diga: "Has hecho clic en el botón con ID: [ID del botón]", usando el valor de data-id.
3. Usa e.target.dataset.id para obtener el ID del botón que se ha clicado.

```
<button data-id="1">Botón 1</button>
```

```
<button data-id="2">Botón 2</button>
```

```
<button data-id="3">Botón 3</button>
```

```
<button data-id="4">Botón 4</button>
```

Ejercicio 2: Habilitar/Deshabilitar Botón según Contenido de Campo

Problema: Deshabilitar un botón si el campo de texto está vacío y habilitarlo cuando tenga contenido.

Actividad:

1. Crea un campo de texto (input type="text") y un botón inicialmente deshabilitado.
2. Al escribir en el campo, verifica si tiene contenido.
3. Si el campo tiene texto, habilita el botón; si está vacío, deshabilítalo.

<h1>Formulario de Ejemplo</h1>

<input type="text" id="campoTexto" placeholder="Escribe algo...">

<button id="btnEnviar" disabled>Enviar</button>

<p id="mensaje"></p>

Ejercicio 3: Cambiar Color de Fondo con Event Delegation

Problema: Crear varios botones, cada uno con un color. Al hacer clic en cualquiera, cambiar el color de fondo de la página al color asignado en el botón.

Actividad:

1. Crea un contenedor con varios botones, cada uno con un atributo data-color que contenga un valor de color.
2. Usa event delegation para capturar los clics en el contenedor.
3. Identifica qué botón fue presionado y cambia el color de fondo del body según el valor de data-color.

<title>Event Delegation - Cambiar Colores</title>

<style>

```
body {  
  transition: background-color 0.5s ease;  
}
```

```
#contenedorBotones {  
  padding: 20px;  
  background-color: white;  
  border-radius: 10px;  
  display: inline-block;  
  margin-top: 50px;  
}
```

```
button {  
  padding: 10px 20px;  
  margin: 5px;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
  font-weight: bold;  
}
```

</style>

<h1>Selecciona un color de fondo</h1>

<div id="contenedorBotones">

```
<button data-color="lightblue">Azul Claro</button>
<button data-color="lightcoral">Coral Claro</button>
<button data-color="lightgreen">Verde Claro</button>
<button data-color="lightpink">Rosa Claro</button>
<button data-color="lightyellow">Amarillo Claro</button>
<button data-color="lavender">Lavanda</button>
</div>

<p id="colorActual">Color actual: Blanco</p>
```

Ejercicio 4: Resaltar Campo en Foco

Problema: Al hacer clic dentro de un campo de texto, resaltarlo con un borde de color. Al salir del campo, eliminar el borde.

Actividad:

1. Crea un campo de texto.
2. Al hacer foco en el campo (focus), aplica un borde de color.
3. Al perder el foco (blur), elimina el borde.

```
<title>Resaltar Campo en Foco</title>
<style>
  body {
    font-family: Arial, sans-serif;
    padding: 20px;
  }
  .campo-estilo {
    padding: 10px;
    margin: 10px 0;
    width: 300px;
    border: 2px solid #ccc;
    border-radius: 5px;
    transition: all 0.3s ease;
  }
  .campo-enfocado {
    border-color: #2196F3;
    box-shadow: 0 0 8px rgba(33, 150, 243, 0.3);
    outline: none;
  }
  .instrucciones {
    color: #666;
    font-size: 14px;
  }
</style>
<h1>Formulario de Práctica</h1>

<p class="instrucciones">Haz clic en cada campo para ver cómo se resalta</p>
```

```

<div>
  <label for="nombre">Nombre:</label><br>
  <input type="text" id="nombre" class="campo-estilo" placeholder="Escribe tu nombre">
</div>

<div>
  <label for="email">Email:</label><br>
  <input type="email" id="email" class="campo-estilo"
placeholder="ejemplo@correo.com">
</div>

<div>
  <label for="mensaje">Mensaje:</label><br>
  <textarea id="mensaje" class="campo-estilo" placeholder="Escribe tu mensaje..."
rows="4"></textarea>
</div>

```

Ejercicios de Propagación y Delegación de Eventos

Ejercicio 5: Evitar Propagación con e.stopPropagation()

Problema: Descubre cómo usar e.stopPropagation() para controlar la propagación de eventos en JavaScript.

Actividad:

1. Crea un contenedor div que tenga dentro un botón.
2. Agrega un evento de clic al div que muestre un mensaje en consola.
3. Agrega un evento de clic al botón que muestre un mensaje diferente, y usa e.stopPropagation() para evitar que el clic en el botón también dispare el evento del div contenedor.

```
<title>Evitar Propagación de Eventos</title>
```

```
<style>
```

```

#contenedorExterno {
  padding: 30px;
  background-color: #e3f2fd;
  border: 3px solid #1976d2;
  border-radius: 10px;
  margin: 20px;
}

```

```

#contenedorInterno {
  padding: 30px;
  background-color: #fce4ec;
}

```

```

border: 3px solid #c2185b;
border-radius: 10px;
margin: 20px;
}

#botonInterno {
padding: 15px 30px;
background-color: #4caf50;
color: white;
border: none;
border-radius: 5px;
font-size: 16px;
cursor: pointer;
}

#botonSinStop {
padding: 15px 30px;
background-color: #ff9800;
color: white;
border: none;
border-radius: 5px;
font-size: 16px;
cursor: pointer;
margin-top: 10px;
}

#consola {
background-color: #f5f5f5;
border: 1px solid #ddd;
padding: 15px;
margin-top: 20px;
font-family: monospace;
min-height: 100px;
border-radius: 5px;
}

.mensaje-contenedor {
color: #1976d2;
font-weight: bold;
}

.mensaje-boton {
color: #c2185b;
font-weight: bold;
}
</style>
<h1>Demostración de stopPropagation()</h1>

```

```
<div id="contenedorExterno">
  <p>Contenedor Externo (Azul)</p>

  <div id="contenedorInterno">
    <p>Contenedor Interno (Rosa)</p>
    <button id="botonInterno">Botón con stopPropagation()</button>
    <button id="botonSinStop">Botón SIN stopPropagation()</button>
  </div>
</div>

<h3>Registro de Eventos:</h3>
<div id="consola">
  <p>Los eventos aparecerán aquí...</p>
</div>
```

Ejercicio 6: Crear Lista de Tareas con data-id y remove()

Problema: Aprende a eliminar tareas de una lista utilizando data-id y remove().

Actividad:

1. Crea una lista en HTML con algunas tareas y un botón "Eliminar" junto a cada tarea. Cada tarea debe tener un atributo data-id único.
2. Al hacer clic en "Eliminar", identifica la tarea usando e.target.dataset.id y elimina solo esa tarea del DOM.
3. Asegúrate de que el evento solo afecte al botón "Eliminar".

```
<title>Lista de Tareas Interactiva</title>
```

```
<style>
```

```
  body {
    font-family: Arial, sans-serif;
    max-width: 600px;
    margin: 0 auto;
    padding: 20px;
  }
```

```
  #formularioTarea {
    margin-bottom: 20px;
    display: flex;
    gap: 10px;
  }
```

```
  #nuevaTarea {
    flex: 1;
    padding: 10px;
    border: 2px solid #ddd;
    border-radius: 4px;
  }
```

```
#agregarTarea {
  padding: 10px 20px;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

#agregarTarea:hover {
  background-color: #45a049;
}

#listaTareas {
  list-style-type: none;
  padding: 0;
}

.tarea {
  background-color: #f9f9f9;
  margin: 8px 0;
  padding: 15px;
  border-radius: 5px;
  border-left: 5px solid #2196F3;
  display: flex;
  justify-content: space-between;
  align-items: center;
  transition: all 0.3s ease;
}

.tarea:hover {
  background-color: #f1f1f1;
  transform: translateX(5px);
}

.tarea.completada {
  opacity: 0.7;
  border-left-color: #4CAF50;
  text-decoration: line-through;
}

.boton-tarea {
  display: flex;
  gap: 5px;
}

.btn-completar, .btn-eliminar {
```

```

padding: 5px 10px;
border: none;
border-radius: 3px;
cursor: pointer;
font-size: 12px;
}

.btn-completar {
background-color: #2196F3;
color: white;
}

.btn-eliminar {
background-color: #f44336;
color: white;
}

.contador {
margin-top: 20px;
padding: 10px;
background-color: #e8f5e8;
border-radius: 5px;
font-weight: bold;
}
</style>
<h1>Gestor de Tareas</h1>

<div id="formularioTarea">
  <input type="text" id="nuevaTarea" placeholder="Escribe una nueva tarea...">
  <button id="agregarTarea">+ Agregar</button>
</div>

<ul id="listaTareas">
  <!-- Las tareas se agregarán dinámicamente -->
</ul>

<div class="contador" id="contadorTareas">
  Tareas pendientes: 0
</div>

```

Ejercicios de Manipulación del DOM

Ejercicio 7: Cambiar Color al Presionar y Soltar

Problema: Mostrar un div de color rojo. Cuando se presione el botón del mouse cambiar el color a amarillo y al soltarlo volver a rojo.

Actividad:

1. Crea un div con color de fondo rojo.
2. Al presionar el botón del mouse sobre el div (mousedown), cambia su color a amarillo.
3. Al soltar el botón (mouseup), restaura el color rojo.

<title>Presiona y Cambia Color</title>

<style>

```
body {  
  font-family: Arial, sans-serif;  
  text-align: center;  
  padding: 50px;  
}
```

```
h1 {  
  color: #333;  
}
```

```
.instrucciones {  
  color: #666;  
  margin-bottom: 30px;  
}
```

```
.mi-div {  
  width: 300px;  
  height: 200px;  
  margin: 20px auto;  
  border-radius: 10px;  
  background-color: red;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  color: white;  
  font-size: 20px;  
  font-weight: bold;  
  cursor: pointer;  
  border: 3px solid darkred;  
  transition: background-color 0.2s;  
  box-shadow: 0 4px 8px rgba(0,0,0,0.2);  
}
```

```
.mi-div:hover {  
  box-shadow: 0 6px 12px rgba(0,0,0,0.3);  
}
```

```
.texto-estado {  
  margin-top: 20px;
```

```
        font-weight: bold;
    }
</style>
<h1>Ejercicio 7: Presiona y Cambia Color</h1>

<p class="instrucciones">
    Presiona el botón del mouse sobre el cuadro rojo<br>
    Mientras mantengas presionado, se pondrá amarillo<br>
    Cuando sueltes, volverá a rojo
</p>

<div class="mi-div" id="miDiv">
    PRESIONA AQUÍ CON EL MOUSE
</div>

<p class="texto-estado" id="estado">Estado: No presionado</p>
```

Ejercicio 8: Cambiar Estilos al Pasar el Mouse

Problema: Mostrar un elemento h1, cambiar su color de texto a rojo y fondo a amarillo al pasar el mouse sobre él, y restaurar estilos al salir.

Actividad:

1. Crea un título h1.
2. Al pasar el mouse sobre él (mouseover), cambia color de texto a rojo y fondo a amarillo.
3. Al quitar el mouse (mouseout), restaura los colores originales.

```
<h1 id="tituloPrincipal">Pasa el mouse aquí</h1>
```

Ejercicio 9: Editor de Texto Interactivo con JavaScript

Problema: Aprende a manipular elementos del DOM seleccionándolos por su ID único y modificando su contenido dinámicamente.

Actividad:

Crea una aplicación interactiva que permita:

1. Seleccionar un elemento h1 por su ID único
2. Modificar su contenido de texto mediante diferentes opciones
3. Implementar funcionalidades para:
 - Cambiar el texto a un valor predeterminado
 - Restaurar el texto original
 - Usar un texto personalizado
 - Permitir al usuario ingresar su propio texto

Elementos HTML proporcionados:

```
<h1 id="miEncabezado">Texto Original</h1>
```

```
<div style="text-align: center; margin-top: 30px;">  
  <button id="btnCambiar">Cambiar Texto</button>  
  <button id="btnRestaurar">Restaurar Original</button>  
  <button id="btnPersonalizado">Mi Texto</button>  
</div>
```

```
<div style="text-align: center; margin-top: 20px;">  
  <input type="text" id="inputTexto" placeholder="Escribe tu texto aquí" style="padding:  
8px; width: 250px;">  
  <button id="btnUsarInput">Usar este Texto</button>  
</div>
```

Ejercicio 10: Seleccionar por ID y Cambiar Estilo

Problema: Aprende a seleccionar un elemento específico de la página por su ID y cambiar su apariencia visual modificando estilos directamente con JavaScript.

Actividad:

Crea una página con un párrafo y varios botones que permitan:

1. Seleccionar un párrafo por su ID único
2. Cambiar su color de fondo usando diferentes colores
3. Restaurar el color original cuando sea necesario

HTML de partida:

```
<p id="miParrafo">Este es un párrafo de ejemplo. Haz clic en los botones para cambiar mi  
color de fondo.</p>
```

```
<div>  
  <button id="btnRojo">Rojo</button>  
  <button id="btnAzul">Azul</button>  
  <button id="btnVerde">Verde</button>  
  <button id="btnAmarillo">Amarillo</button>  
  <button id="btnRestaurar">Color Original</button>  
</div>
```

Ejercicio 11: Encontrar y Resaltar Elementos

Problema: Aprende a seleccionar elementos por su clase CSS usando `querySelector()`. Encuentra el primer elemento con una clase específica y modifica su apariencia visual.

Actividad:

Crea una página con varios elementos que compartan la misma clase y:

1. Usa `querySelector()` para encontrar el PRIMER elemento con una clase específica
2. Añade diferentes tipos de bordes para resaltarlo
3. Practica seleccionando diferentes elementos por su clase

HTML de partida:

```
<div class="caja">Caja 1</div>
```

```
<div class="caja">Caja 2</div>
```

```
<div class="caja">Caja 3</div>
```

```
<div class="caja">Caja 4</div>
```

```
<button id="btnBordeRojo">Borde Rojo al Primero</button>
```

```
<button id="btnBordeAzul">Borde Azul al Primero</button>
```

```
<button id="btnBordeVerde">Borde Verde al Primero</button>
```

```
<button id="btnQuitarBorde">Quitar Borde</button>
```

Ejercicio 12: Modificar Varios Elementos a la Vez

Problema: Aprende a seleccionar TODOS los elementos que comparten una misma clase usando `querySelectorAll()` y aplicar cambios a cada uno de ellos.

Actividad:

Crea una lista de elementos y:

1. Usa `querySelectorAll()` para seleccionar TODOS los elementos con una clase específica
2. Recorre la lista de elementos usando un bucle
3. Aplica cambios de color a cada elemento de la lista

HTML de partida:

```
<ul>
```

```
  <li class="item-lista">Elemento 1</li>
```

```
  <li class="item-lista">Elemento 2</li>
```

```
  <li class="item-lista">Elemento 3</li>
```

```
  <li class="item-lista">Elemento 4</li>
```

```
</ul>
```

```
<button id="btnRojo">Texto Rojo</button>
```

```
<button id="btnAzul">Texto Azul</button>
```

```
<button id="btnVerde">Texto Verde</button>
```

```
<button id="btnOriginal">Color Original</button>
```

Ejercicio 13: Seleccionar por Etiqueta y Cambiar Atributo

Problema: Aprende a seleccionar elementos por su etiqueta HTML y modificar sus atributos para cambiar contenido visual de una página.

Actividad:

Crea una galería simple donde:

1. Selecciones una imagen por su ID
2. Cambies su imagen (atributo src)
3. Modifique su texto alternativo (atributo alt)
4. Practiques con diferentes tipos de imágenes

HTML de partida:

```

```

```
<button id="btnImagen1">Mostrar Gato</button>
```

```
<button id="btnImagen2">Mostrar Perro</button>
```

```
<button id="btnImagen3">Mostrar Pájaro</button>
```

```
<button id="btnRestaurar">Imagen Original</button>
```

Ejercicio 14: Encontrar Elementos Dentro de Contenedores

Problema: Aprende a seleccionar elementos que están dentro de otros elementos específicos usando selectores CSS avanzados en JavaScript.

Actividad:

Crea una estructura HTML con contenedores y:

1. Usa selectores descendientes para encontrar párrafos dentro de divs específicos
2. Añade clases CSS a esos elementos usando `classList.add()`
3. Practica diferentes formas de seleccionar elementos anidados

HTML de partida:

```
<div class="contenedor">
```

```
  <p>Este párrafo está dentro del contenedor</p>
```

```
</div>
```

```
<div class="contenedor-especial">
```

```
  <p>Este párrafo está dentro del contenedor especial</p>
```

```
</div>
```

```
<button id="btnResaltar">Resaltar Párrafo</button>
```

```
<button id="btnQuitar">Quitar Resaltado</button>
```

Ejercicio 15: Capturar y Mostrar Texto del Usuario

Problema: Aprende a obtener el texto que un usuario escribe en un campo de entrada y mostrarlo en otra parte de la página.

Actividad:

Crea un formulario simple donde:

1. El usuario escriba texto en un campo de entrada (input)
2. Al hacer clic en un botón, captures ese texto
3. Muestres el texto capturado en un párrafo o elemento de la página

HTML de partida:

```
<input type="text" id="miInput" placeholder="Escribe algo...">
<button id="miBoton">Mostrar Texto</button>
<p id="resultado">Aquí aparecerá tu texto</p>
```

Ejercicio 16: Encontrar y Modificar Elementos Específicos en una Lista

Problema: Aprende a seleccionar elementos específicos dentro de una lista usando selectores especiales como :nth-child().

Actividad:

Crea una lista de elementos y:

1. Usa :nth-child() para seleccionar elementos específicos por su posición
2. Modifica el contenido de esos elementos específicos
3. Practica seleccionando diferentes posiciones en la lista

HTML de partida:

```
<ul>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
  <li>Elemento 4</li>
</ul>

<button id="btnPrimero">Modificar Primero</button>
<button id="btnSegundo">Modificar Segundo</button>
<button id="btnTercero">Modificar Tercero</button>
<button id="btnUltimo">Modificar Último</button>
```

Ejercicio 17: Buscar Elementos Dentro de un Contenedor Específico

Problema: Aprende a combinar diferentes métodos de selección para encontrar elementos específicos dentro de una sección concreta de la página.

Actividad:

Crea una estructura HTML con contenedores y elementos internos, y:

1. Primero selecciona un contenedor principal por su ID
2. Luego busca elementos específicos DENTRO de ese contenedor usando clases
3. Modifica solo los elementos dentro del contenedor seleccionado

HTML de partida:

```
<div id="contenedorPrincipal">
  <p class="item">Texto 1</p>
  <p class="item">Texto 2</p>
  <p class="item">Texto 3</p>
</div>
```

```
<div id="otroContenedor">
  <p class="item">Texto A</p>
  <p class="item">Texto B</p>
</div>
```

```
<button id="btnModificar">Modificar Items del Contenedor</button>
```

Ejercicio 18: Modificar Elementos Relacionados Dinámicamente

Problema: Aprende a crear funciones que modifiquen elementos específicos relacionados con el botón que se presiona, usando los métodos `closest()` y `querySelector()`.

Actividad:

Crea una página con varias secciones independientes, cada una con:

1. Un título
2. Un párrafo de contenido
3. Un botón "Modificar"

Implementa una función que:

1. Al hacer clic en cualquier botón "Modificar"
2. Encuentre la sección padre correspondiente usando `closest()`
3. Busque dentro de esa sección el párrafo de contenido usando `querySelector()`
4. Modifique solo el contenido de esa sección específica

Objetivos de aprendizaje:

- ✓ Entender cómo usar `closest()` para navegar hacia elementos padres
- ✓ Aprender a buscar elementos dentro de un contenedor específico
- ✓ Crear funciones reutilizables que trabajen con elementos relacionados
- ✓ Entender el uso de `this` para referenciar el elemento actual

HTML de partida sugerido:

```
<div class="seccion">
  <h3>Título Sección 1</h3>
  <p class="contenido">Contenido de la sección 1</p>
  <button class="btn-modificar">Modificar</button>
</div>
```

```
<div class="seccion">
  <h3>Título Sección 2</h3>
  <p class="contenido">Contenido de la sección 2</p>
  <button class="btn-modificar">Modificar</button>
</div>
```

Requisitos técnicos:

1. Usar `closest('.seccion')` para encontrar la sección padre
2. Usar `querySelector('.contenido')` dentro de la sección encontrada
3. Modificar solo el contenido de la sección donde se hizo clic
4. Cada botón debe funcionar independientemente

Resultado esperado:

- Al hacer clic en el botón de la Sección 1 → Solo se modifica la Sección 1
- Al hacer clic en el botón de la Sección 2 → Solo se modifica la Sección 2
- Las demás secciones permanecen sin cambios

Preguntas para reflexionar:

1. ¿Por qué usar `closest()` en lugar de `getElementById()`?
 2. ¿Qué ventaja tiene buscar con `querySelector()` dentro de un elemento específico?
 3. ¿Cómo se asegura que cada botón modifique solo su sección correspondiente?
-

Ejercicios Adicionales Propuestos

Ejercicio 19: Crear un Contador Interactivo de Clics

Problema: Aprende a crear un contador que registre y muestre cuántas veces se ha hecho clic en un botón.

Actividad:

Crea una aplicación web simple que incluya:

1. Un botón para hacer clic
2. Un elemento que muestre el número de clics realizados
3. Implementa la lógica para incrementar el contador con cada clic
4. Actualiza la pantalla con el nuevo valor del contador

Objetivos de aprendizaje:

- ✓ Crear y manipular variables para almacenar datos
- ✓ Conectar eventos de clic a funciones JavaScript
- ✓ Actualizar contenido HTML dinámicamente
- ✓ Entender el concepto de estado en una aplicación web

HTML de partida sugerido:

```
<button id="botonClic">Haz clic aquí</button>
<p id="contador">Clics: 0</p>
```

Ejercicio 20: Validación de Formulario en Tiempo Real

Problema: Aprende a validar información del usuario mientras la escribe, proporcionando retroalimentación inmediata.

Actividad:

Crea un formulario con un campo de email y:

1. Escucha el evento input para detectar cada vez que el usuario escribe
2. Valida si el email tiene un formato correcto
3. Muestra mensajes de ayuda al usuario en tiempo real
4. Cambia los estilos visuales según si el email es válido o no

Objetivos de aprendizaje:

- ✓ Usar el evento input para detectar cambios en tiempo real
- ✓ Crear funciones de validación para formatos específicos
- ✓ Proporcionar retroalimentación visual inmediata al usuario
- ✓ Mejorar la experiencia de usuario con validación instantánea

HTML de partida sugerido:

```
<input type="email" id="emailInput" placeholder="ejemplo@correo.com">
<div id="mensajeEmail"></div>
<button id="btnEnviar">Enviar</button>
```

Ejercicio 21: Crear una Galería de Imágenes Interactiva

Problema: Aprende a crear una galería donde las imágenes en miniatura al hacer clic muestren una versión grande, usando atributos personalizados y event delegation.

Actividad:

Crea una galería con:

1. Un área principal para mostrar la imagen grande
2. Una fila de miniaturas (imágenes pequeñas)
3. Cada miniatura debe tener un atributo data-large con la URL de su versión grande
4. Al hacer clic en cualquier miniatura, mostrar su versión grande en el área principal

Objetivos de aprendizaje:

- ✓ Usar atributos data-* para almacenar información adicional
- ✓ Implementar event delegation para manejar múltiples elementos
- ✓ Cambiar atributos src de imágenes dinámicamente
- ✓ Crear interfaces interactivas de galería

HTML de partida sugerido:

```
<div id="imagenGrande">
  <img src="" alt="Imagen seleccionada" id="imagenPrincipal">
</div>
```

```
<div id="galeriaMiniaturas">
  
```

```


</div>
```

Ejercicio 22: Crear un Acordeón Interactivo (Expandir/Contraer)

Problema: Aprende a crear un componente acordeón donde las secciones se expanden y contraen al hacer clic, mostrando u ocultando contenido adicional.

Actividad:

Crea un componente acordeón con:

1. Múltiples secciones, cada una con un título clickeable
2. Contenido que está inicialmente oculto
3. Funcionalidad para expandir/contraer al hacer clic en el título
4. Comportamiento donde solo una sección puede estar expandida a la vez

Objetivos de aprendizaje:

- ✓ Crear interfaces colapsables/expandibles
- ✓ Manipular clases CSS para mostrar/ocultar elementos
- ✓ Implementar lógica para mantener solo un elemento activo
- ✓ Mejorar la experiencia de usuario con interacciones suaves

HTML de partida sugerido:

```
<div class="acordeon">
  <div class="seccion">
    <h3 class="titulo">Título Sección 1</h3>
    <div class="contenido">Contenido de la sección 1...</div>
  </div>
  <div class="seccion">
    <h3 class="titulo">Título Sección 2</h3>
    <div class="contenido">Contenido de la sección 2...</div>
  </div>
</div>
```

Pasos a seguir:

1. Preparar estructura HTML:
 - Contenedor principal .acordeon
 - Varias secciones .seccion
 - Títulos clickeables .titulo
 - Contenido oculto inicialmente .contenido
2. Configurar estilos CSS:
 - Ocultar contenido por defecto (display: none o max-height: 0)
 - Estilos para títulos (cursor pointer, hover effects)
 - Clase .activa para secciones expandidas
3. Implementar funcionalidad JavaScript:
 - Escuchar clics en los títulos
 - Alternar visibilidad del contenido de la sección clickeada
 - Cerrar otras secciones al abrir una nueva

- Actualizar indicadores visuales (iconos, colores)
- 4. Agregar transiciones:
 - Animaciones suaves al expandir/contraer
 - Cambios de color o íconos indicativos

Requisitos técnicos:

- Solo una sección puede estar expandida a la vez
- Contenido inicialmente oculto
- Transiciones suaves al expandir/contraer
- Títulos claramente clickeables (cursor: pointer)

Resultado esperado:

- Al cargar: todas las secciones contraídas
- Clic en sección 1: expande sección 1, otras cerradas
- Clic en sección 2: expande sección 2, cierra sección 1
- Clic en sección expandida: la cierra

Preguntas para reflexionar:

1. ¿Por qué es útil que solo una sección esté expandida a la vez?
 2. ¿Cómo podríamos modificar el código para permitir múltiples expandidas?
 3. ¿Qué ventajas tiene un acordeón sobre mostrar todo el contenido a la vez?
 4. ¿Cómo mejoraríamos la accesibilidad para usuarios con teclado?
-

Preguntas Frecuentes

Sobre Eventos

1. ¿Por qué usar `addEventListener` en lugar de atributos HTML como `onclick`?
 - Permite separar la lógica JavaScript del HTML.
 - Permite manejar múltiples eventos del mismo tipo sin sobrescribirlos.
2. ¿Cómo obtener datos del evento (coordenadas, tecla presionada)?
 - El objeto evento contiene propiedades como `clientX`, `clientY`, `key`, etc.
3. ¿Cómo evitar la acción por defecto de un evento?
 - Usar `evento.preventDefault()` para cancelar acciones predeterminadas.

Sobre Selección de Elementos

1. ¿Cuál es la principal diferencia entre `getElementById` y `querySelector`?
 - `getElementById` busca solo por ID y es más rápido para eso.
 - `querySelector` usa cualquier selector CSS y es más flexible.
2. ¿Qué devuelve `querySelectorAll` y cómo se usa?

- Devuelve una `NodeList` con todos los elementos coincidentes.
 - Se puede iterar con `forEach` o bucles.
3. ¿Cuándo usar cada método de selección?
- `getElementById`: Para IDs únicos.
 - `querySelector`: Para seleccionar el primer elemento con cualquier selector.
 - `querySelectorAll`: Para seleccionar todos los elementos con un selector.
-

Nota: Para cada ejercicio, se recomienda crear archivos HTML y JavaScript separados, siguiendo las mejores prácticas de estructura y organización de código.